

RESEARCH ARTICLE

SP-RTSD: A Lightweight Real-Time Strawberry Detection on Edge Devices for Onboard Robotic Harvesting

Yujie Chen¹ | Aijing Shu¹ | Zhanhao Liu¹ | Yang Chen^{2,3} | Won Suk Lee³ | Yanchao Zhang¹ 

¹School of Information Science and Engineering, Zhejiang Sci-Tech University, Hangzhou, Zhejiang, China | ²College of Computer Science and Technology, Zhejiang University, Hangzhou, Zhejiang, China | ³Department of Agricultural and Biological Engineering, University of Florida, Gainesville, Florida, USA

Correspondence: Yanchao Zhang (yczhang@zstu.edu.cn)

Received: 11 May 2024 | **Revised:** 9 October 2024 | **Accepted:** 18 April 2025

Funding: This study was funded by the National Natural Science Foundation of China (NSFC), grant number: 61905219, the Zhejiang Science and Technology Cooperation Plan, grant number: 2024SNJF071, China Agriculture Research System (CARS-01), and National Key Research and Development Program of China (2023YFD1401100).

Keywords: edge devices | lightweight | real-time detection | RTSD | SP-RTSD

ABSTRACT

On-board strawberry-picking robots offer the potential to significantly reduce labor costs and enhance picking efficiency. How to achieve high precision and fast strawberry recognition on resource-constrained edge devices is the key to robotic strawberry harvesting. Before developing our model, two lightweighting methods that maintain model structure are explored to substantiate the thesis that only judicious compression strategies tailored to edge hardware specifications can transform heavyweight deep models into efficient and compact deployments with enhanced performance on embedded devices. Based on this, and in combination with RTSD, Superb Real-time Strawberry Detection (SP-RTSD), which is designed to achieve faster and more accurate strawberry recognition on edge devices. Firstly, the C2f-Faster module performs channel-wise feature screening to enhance feature extraction efficiency while reducing model parameters; secondly, a lightweight recognition head with a parameter sharing mechanism is proposed specially for the edge devices. The speed of SP-RTSD was significantly improved by 22% from 20.63 to 25.18 FPS, which is similar to the 25.2 FPS of RTSD. Without changing the model structure, the model size is reduced by 40.3% from 6.2 to 3.7 MB. In contrast to typical lightweight strategies, which often boost inference speed at the cost of accuracy, SP-RTSD achieves exceptional accuracy with a mean average precision (mAP) of 91.7%, slightly outperforming the original baseline model (90.7%). The improvements in accuracy, speed, and size demonstrate that SP-RTSD addresses the challenge of balancing accuracy with inference speed on edge devices. In comparison experiments with other advanced object detection and lightweight models, as well as tests on additional open-source strawberry data sets, SP-RTSD consistently delivered superior results, affirming its robustness. Furthermore, SP-RTSD demonstrated an impressive combined success rate of 92% in strawberry grasping simulation experiments with a robotic arm, thereby confirming its suitability for integration into practical picking machines.

1 | Introduction

Strawberries, characterized by their richness in vitamin C and high nutritional value, exhibit extensive applications in the realms of food, medicine, and cosmetics. In recent years, the demand for strawberries has witnessed a steady increase, prompting a corresponding expansion in both planting areas and production. It is

imperative to acknowledge the inherent challenge posed by the brief ripening period of strawberries, where untimely picking can lead to fruit rot and subsequent economic losses. Presently, the conventional reliance on manual methods for strawberry picking exacerbates the issue of untimely harvesting. However, the advent of smart agriculture has drawn considerable attention to robotic picking, driven by its notable advantages in efficiency, time

efficiency, and cost-effectiveness. Particularly noteworthy is the critical role that accurate strawberry detection plays in the advancement of automated robotic picking technologies.

The target detection algorithm serves as a real-time technique for identifying and tracking targets through algorithmic modeling. Traditional methods for target detection often rely on manually designed feature extraction and classifiers, involving the traversal of image data sets, feature extraction, and training classifiers to identify target images. Anraeni et al. (2021). For instance, RGB feature extraction and the k-Nearest Neighbors Algorithm (k-NN) were employed to recognize strawberry fruit ripening. Lin et al. (2020), on the other hand, utilized profile information for partial matching to obtain candidate fruits and applied a probabilistic Hough transform, followed by an SVM classifier for fruit recognition. Rizon et al (Rizon et al. 2015) introduced a mango fruit detection method based on Hough transform, specifically designed to detect partially occluded and overlapped mango fruits by extracting texture features. Despite the efficacy of traditional target detection methods in solving certain problems, they often encounter challenges, such as poor robustness, high computational complexity, and diminished accuracy in practical applications.

Compared to traditional machine vision techniques, deep learning approaches significantly enhance the accuracy and speed of object detection in intricate environments by extracting more intricate features from image data (He et al. 2022). Bai et al. (2024) introduced the Swin Transformer detection head onto the YOLOv7 framework, enhancing the model's spatial interaction capabilities and feature extraction in scenes with similar colors and overlapping or obscured elements. This modification resulted in notable improvements, achieving recognition accuracies of 93.9% for strawberries and 91.3% for flowers and fruits. Li et al. (2024) proposed the YOLOv7-Litchi method, which excels in detecting densely distributed lychee fruits that often occlude each other in complex backgrounds. This approach demonstrated superior detection abilities, culminating in an accuracy rate of 94.6% and an impressive mean average precision (mAP) value of 98.60%. Despite these advancements, the practical applications of agricultural machinery often demand swift task execution on edge devices. Consequently, researchers are increasingly focused on the development of lightweight deep neural networks. This emphasis on lightweight models aims to strike a balance between computational efficiency and accuracy, catering to the real-time requirements of tasks in agricultural settings.

Wang and He (2021). innovatively developed a channel-pruned YOLOV5s model using transfer learning for rapid apple fruit detection before thinning. This approach significantly reduced the number of parameters by 92.7%, resulting in a model size of merely 9.5% of the original. Notably, this method successfully streamlines the model architecture while ensuring accurate precision. Pang and Chen (2023) introduced the MS-YOLOv5 model, incorporating depth hybrid deformable convolution, a dual synergetic attention mechanism, and swift fusion of cross-scale weights. On the strawberry data set, this model demonstrated an impressive improvement, with 8.4% higher mAP and 1.3% higher frames per second (FPS) than the original network, accompanied by a 45.8% reduction in model size. However, it is noteworthy that many lightweight approaches tend to prioritize reducing the model parameter count over achieving optimal performance results on edge devices. Zhang

et al. (2022) proposed RTSD algorithm based on YOLOv4-tiny. Through the reduction of CBL modules and convolutional layers in the CSPNet backbone, the network achieved a detection speed of 25.2 FPS on edge devices, enabling real-time strawberry fruit detection. However, this approach comes with a slight sacrifice in accuracy, achieving a final accuracy of 82.25%, falling short of the practical detection accuracy requirements. In another study, Zhou et al. (2023) designed a lightweight backbone utilizing the VGG architecture, resulting in a remarkable 76.35% reduction in parameters and a 38.75% reduction in inference time compared to the original model. Despite these improvements, the actual inference time when deployed to edge devices reached 926 ms, revealing a considerable gap from meeting the real-time detection application requirements at edge devices.

To address the above issues, this paper introduces a lightweight detection algorithm designed specifically for strawberry detection on edge devices, with the goal of enhancing both recognition accuracy and detection efficiency for strawberry-picking robots deployed on edge boards. The key contributions of this study are as follows:

- (1) We investigate the reasons behind the failure of certain lightweighting strategies during real-world deployment on edge devices, offering valuable insights that can guide future efforts in lightweight algorithm design for embedded systems.
- (2) We propose the C2f-Faster module, which effectively extracts spatial features and channel information, significantly improving the accuracy of strawberry identification, particularly in complex environments with occlusion and shadows.
- (3) We develop the Lw-Detect, a lightweight detection head with a parameter-sharing mechanism, which substantially boosts the strawberry detection rate while maintaining high accuracy levels.
- (4) The SP-RTSD model successfully balances both accuracy and inference speed, addressing the critical challenge of optimizing these factors for deployment on edge devices.
- (5) Through a series of ablation studies, comparative model evaluations, data set validation experiments, and robotic simulation trials, we demonstrate the superiority and robustness of the SP-RTSD model, confirming its suitability for real-world applications in strawberry-picking robots.

2 | Materials and Methods

2.1 | System Setup

As illustrated in Figure 1, our integrated system design encompasses four principal phases. Initially, an unmanned aerial vehicle (UAV) was employed to collect data from strawberry fields. Subsequently, a series of lightweight exploratory experiments were conducted with the objective of improving the efficiency of the edge devices. The objective of these experiments was to identify strategies to enhance accuracy while maintaining the system's lightweight operational profile. The insights gained from these experiments informed the development of the SP-RTSD model, which is designed for deployment

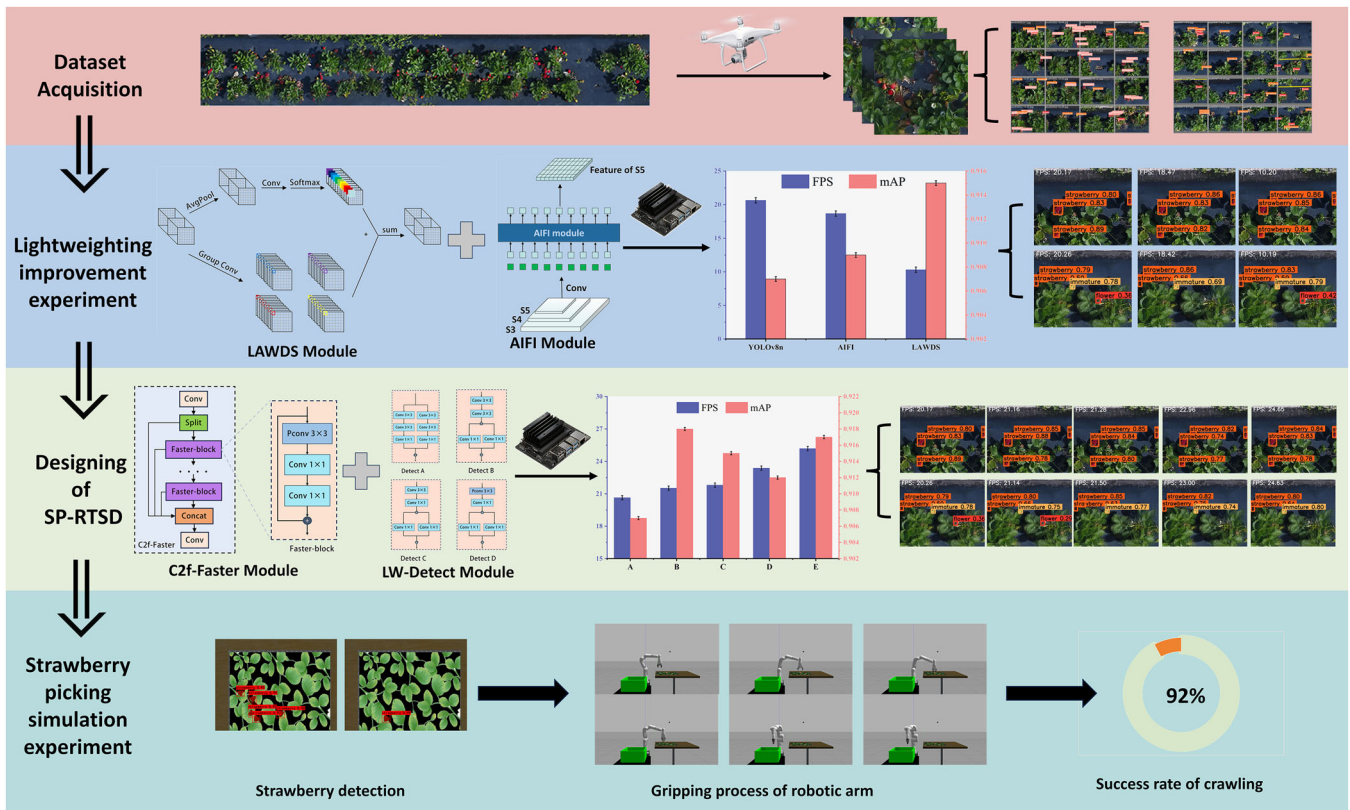


FIGURE 1 | System technology flowchart of real-time strawberry detection.

on edge devices. This innovative model demonstrated a notable increase in the number of FPS compared to the baseline, while simultaneously enhancing accuracy and meeting real-time detection standards. Finally, simulation experiments using the integrated robotic arm achieved an impressive 92% success rate in capturing single strawberries and composite scenes of multiple strawberries. These experiments further validated the efficacy and superiority of the designed SP-RTSD model.

2.2 | Image Acquisition and Processing

The strawberry data set utilized in this study was sourced from a strawberry field established by the Plant Science Research and Education Unit (PSREU) at the University of Florida, Citra, Florida, USA (coordinates: 29.404265° N, 82.141893° W). Data collection was conducted using a DJI Phantom 4 Pro UAV (DJI Technology Co. Ltd., Shenzhen, China), with an optimal image acquisition height set at 3 meters to ensure superior recognition. The UAV's capacity to capture a comprehensive overhead view enables the generation of a panoramic image of the fruit-picking scene. This panoramic perspective plays a pivotal role in enhancing the robot's comprehension of the environment and scene dynamics during the fruit-picking task. Post-image acquisition, image segmentation was employed, resulting in 3428 small rectangular strawberry images with dimensions of 480 × 380 pixels. A selection of these strawberry images is presented in Figure 2. Following image acquisition and segmentation, the images underwent labeling using the LabelImg software. Rectangular borders of distinct colors were employed to demarcate three different targets: ripe strawberries, immature strawberries, and flowers. Given the presence of

occluded fruiting flowers within the data set, no further processing was conducted during the labeling process to restore the original scene. The labeled files were then saved in a .txt format. Subsequently, the data set was partitioned into training, validation, and test sets in a proportion of 7:1:2. This meticulous data set preparation ensures the robustness and reliability of the subsequent deep learning model for strawberry detection.

2.3 | Baseline Model

We have chosen the You Only Look Once (YOLO) framework as the baseline model. YOLO, known for its efficiency in achieving fast and accurate object detection in a single stage (Redmon et al. 2016), has evolved significantly from its inception with YOLOv1 to its latest iteration, YOLOv8. In particular, the YOLOv8 series has been widely adopted for real-time target detection and is considered the most advanced algorithm within the YOLO family. YOLOv8 is specifically designed to prioritize improved inference speed while maintaining high target detection accuracy. The YOLOv8 target detection model is characterized by achieving accelerated detection speed, ensuring precise accuracy, and maintaining a more compact model size. The YOLOv8 system consists of five distinct sub-models: YOLOv8-Nano, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x. The primary difference between these submodels is the number of feature extraction modules and convolution kernels at specific locations in the network. In particular, the size of their models and the number of coefficients increase sequentially. For our application on edge devices, we chose the YOLOv8-Nano as a baseline model. Figure 3 illustrates the composition of YOLOv8, showing key modules such as C2f, Conv, Concat, and Upsample.



FIGURE 2 | Strawberry image obtained after image segmentation.

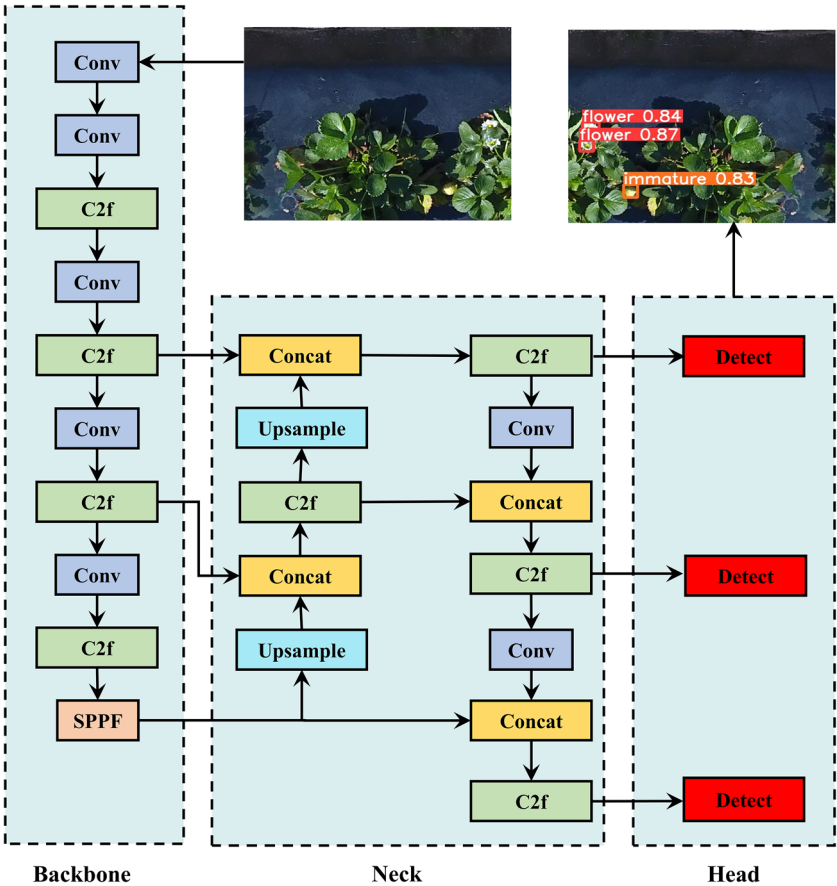


FIGURE 3 | Network structure of original YOLOv8.

2.4 | Lightweighting Improvement Experiment

The current research tends to focus on lightweighting strategies for networks without considering the incorporation of real-world edge device applications. RTSD is proposed as a potential solution to this problem. However, the lightweighting of RTSD is achieved by compressing the model, which sacrifices accuracy. In response to this situation, we explore alternative lightweighting methods that preserve the structure of the model. The objective of this experiment is to identify effective lightweighting improvements for edge devices, particularly those with self-contained hardware acceleration. The replacement of the standard convolution with a more efficient alternative and the introduction of a channel attention mechanism represent effective lightweighting measures that do not necessitate alterations to the model structure. In this experiment, we will validate and explore these two common lightweighting methods. A lightweight model that can be used for real-time strawberry detection is then designed. We have designed LAWDS and AIFI modules for these two lightweighting strategies, and the detailed information is as follows.

2.4.1 | LAWDS Module

The original model's Conv module has been replaced with the LAWDS module, a novel Light Adaptive-weight DownSampling module shown in Figure 4. The LAWDS module is divided into two parts, each contributing to its lightweight design and improved detection accuracy. The first part introduces a lightweight attention mechanism to dynamically adjust input feature weights based on the relationship between global information and channels. The module first obtains the global information of the input through an average pooling layer. It then utilizes this information to perform a linear transformation between channels, which is achieved through a 1×1 convolutional layer. This process allows the module to adaptively assign weights to input features, enhancing its capacity to capture essential information. The second part of the LAWDS module is inspired by YOLOv5's Focus operation, which involves slicing and preserving points on the feature map into channels. However, due to the computational intensity and

time-consuming nature of the Focus operation, we opted for a more efficient alternative. We replaced the Focus operation with convolutional grouping, which allows for additional feature extraction and downsampling while maintaining computational efficiency. Compared to the original Conv module, the LAWDS module is less computationally and parametrically demanding. This modification results in higher detection accuracy, affirming the efficacy of our approach in achieving a more efficient and effective feature extraction and downsampling process.

The LAWDS framework can be divided into three key stages: attention weight computation, downsampling convolution, and weighted summation to generate the final output. Given the input $X \in R^{B \times C \times 2H \times 2W}$, where B denotes the batch size, C the number of channels, and $2H$ and $2W$ the height and width of the feature map, respectively, the attention weight computation proceeds as follows:

$$A = \text{Conv}(\text{AvgPool}(X)) \in R^{B \times C \times 2H \times 2W} \quad (1)$$

$$A_{\text{rearrange}} = \text{rearrange}(A) \in R^{B \times C \times H \times W \times 4} \quad (2)$$

$$w = \text{Softmax}(A_{\text{rearrange}}) \quad (3)$$

where A is the attention graph generated after average pooling and convolution operations, $A_{\text{rearrange}}$ is the rearranged attention graph, and w is the final result of attention weight calculation. The downsampling convolution proceeds as follows:

$$X_{\text{downsampled}} = \text{Conv}_{\text{downsampling}}(X) \in R^{B \times 4C \times H \times W} \quad (4)$$

$$X_{\text{rearrange}} = \text{rearrange}(X_{\text{downsampled}}) \in R^{B \times C \times H \times W \times 4} \quad (5)$$

where $X_{\text{downsampled}}$ is the result after downsampling by the convolution operation and $X_{\text{rearrange}}$ is the result after rearranging, which achieves the same shape as the attention weights. The output Y is ultimately produced by weighted summation:

$$Y = \sum_{i=1}^4 X_{\text{rearrange},i} \times w_i \in R^{B \times C \times H \times W} \quad (6)$$

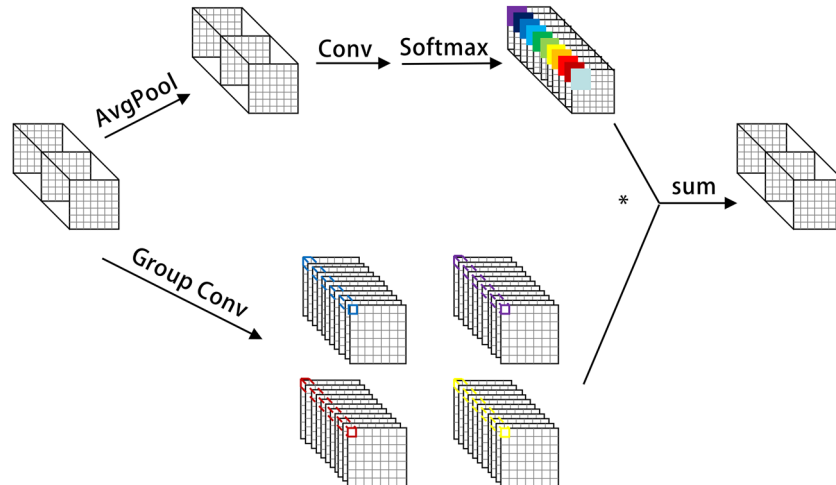


FIGURE 4 | The process of LAWDS implementation.

2.4.2 | AIFI Module

To construct lightweight modules, we replaced the SPPF in the YOLOv8 module with an AIFI module, which was inspired by the RT-DETR model proposed by Baidu (Lv et al. 2023). Please see Figure 5 for a depiction of the implemented module. The AIFI module is based on the use of a self-attention mechanism to handle high-level features in the image. This mechanism enables the model to selectively process specific regions of the data while considering the relevance of other portions. This approach is particularly suitable for processing high-level image features enriched with semantic information. The AIFI module is designed to primarily focus on high-level features, reducing the risk of redundancy and confusion when interacting with low-level features. The self-attention mechanism used by the AIFI module ensures that it processes high-level features without engaging with low-level features at the internal scale. This design choice effectively avoids potential issues related to the interaction of low-level features, such as the risk of repetition and confusion. This contributes to the lightweight nature of our modules.

The whole feature extraction process of AIFI can also be divided into three steps: 2D sine-cosine position embedding computation, Transformer Encoder operation and recovering the shape of the output by taking the input $X \in R^{B \times C \times H \times W}$, where B is the batch size, C is the number of channels, and H and W are the height and the width of the feature map, respectively. The 2D sine-cosine positional embedding is computed as follows:

$$\text{grid}_w, \text{grid}_h = \text{meshgrid}(\text{arrange}(W), \text{arrange}(H)) \quad (7)$$

$$S = \frac{1}{\text{temperature}^{\text{arrange}(C/4)}} \quad (8)$$

$$\sin_out_w = \sin(\text{grid}_w \cdot S), \quad \sin_out_h = \sin(\text{grid}_h \cdot S) \quad (9)$$

$$\cos_out_w = \cos(\text{grid}_w \cdot S), \quad \cos_out_h = \cos(\text{grid}_h \cdot S) \quad (10)$$

$$P = \text{concat}(\sin_out_w, \cos_out_w, \sin_out_h, \cos_out_h) \in R^{H \times W \times C} \quad (11)$$

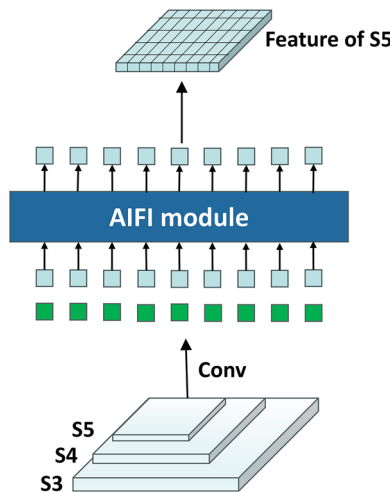


FIGURE 5 | Feature extraction process of AIFI module.

where Equation (7) is the generated 2D grid result of the image, S is the frequency scaling factor of the position embedding matrix, Equations (9) and (10) are the process of calculating the sine and cosine components, and P is the merging of the sine and cosine components to generate the final position embedding matrix result. The Transformer Encoder operation procedure is as follows:

$$X_{\text{flat}} = \text{permute}(\text{flatten}(X)) \in R^{B \times HW \times C} \quad (12)$$

$$X_{\text{transformed}} = \text{Transformer Encoder Layer}(X_{\text{flat}}, P) \quad (13)$$

where Equation (12) is the arithmetic realization of the process of combining with positional embedding after spreading the input features, and Equation (13) is the final result of the Transformer Encoder operation. Finally, the output is transposed and restored to its original shape to obtain the final result Y , as shown in the following Equation:

$$Y = \text{view}(\text{permute}(X_{\text{transformed}})) \in R^{B \times C \times H \times W} \quad (14)$$

2.5 | SP-RTSD Network

After collecting preliminary experimental results on edge devices, we started to develop a lightweight model tailored for real-time strawberry detection. To improve the accuracy and speed of strawberry detection on edge devices, we introduced the newly designed C2f-Faster module and Lw-Detect module. We focus on the two most parameterized modules in the YOLOv8 network: specifically, the C2f-Faster module replaces the original C2f module, while the Lw-Detect module replaces the Detect module. During the construction of these two modules, complex structures such as multi-branching are avoided, the number of parameters is reduced, and the running efficiency of the model is improved. The design principles are closely related to the acceleration strategy for edge devices, with the aim of optimizing the performance of the model specifically for such environments. For convenience, we refer to the enhanced network as the Superb Real-Time Strawberry Detection (SP-RTSD) network, the refined structure of which is shown in Figure 6. In this section, we describe both modules in detail.

2.5.1 | C2f-Faster Module

Edge devices have limited computational power, and if we want to achieve real-time detection based on ensuring that the model structure remains unchanged, using something like Depthwise Separable Convolution instead of standard convolution can be used as a method, which can reduce the number of parameters and computational complexity without affecting the detection accuracy. In FasterNet lightweight network, Chen et al. (2023) proposed a simple PConv (Partial Convolutions) operation, which reduces redundant computations and memory accesses while allowing more efficient extraction of spatial features. We incorporated the PConv to construct the Faster block module, replacing the Bottleneck in C2f and forming the new C2f-Faster module, as illustrated in Figure 7.

Figure 8 outlines the operating principle of PConv, where regular convolution for spatial feature extraction is applied

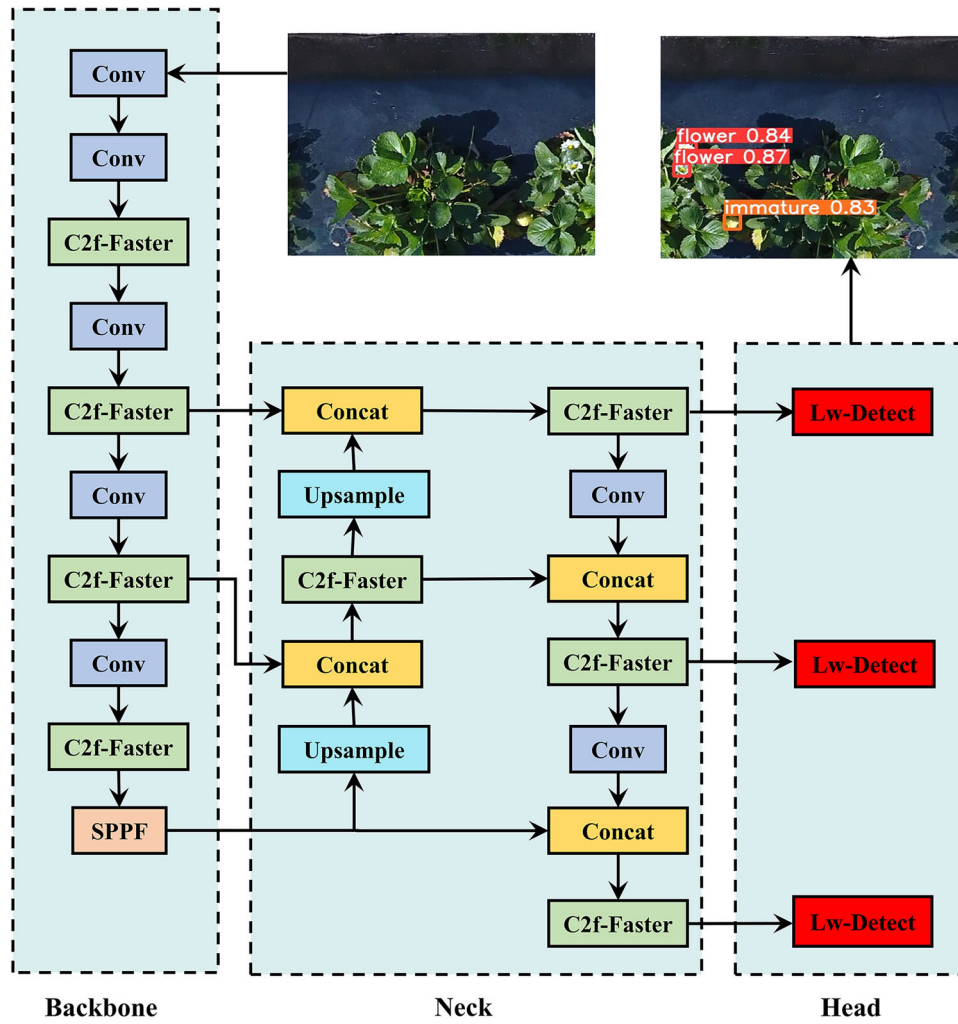


FIGURE 6 | SP-RTSD network structure.

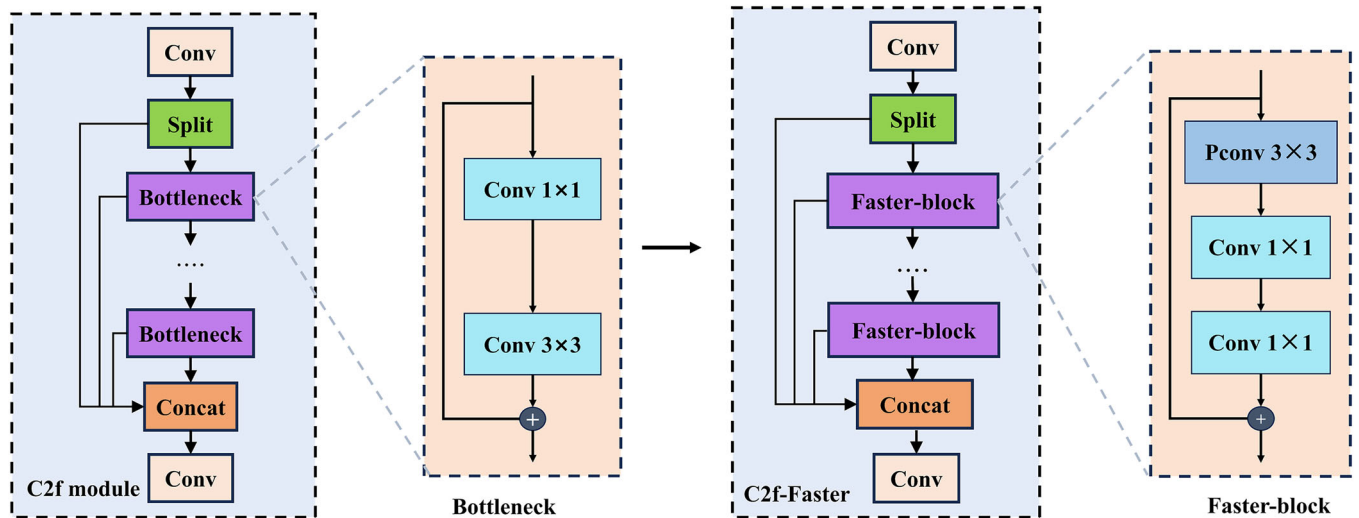


FIGURE 7 | The process of building the C2f-faster module.

only to selected input channels, leaving the rest unchanged. For successive or regular memory accesses, the first or last successive C_p channels are considered as computed representatives of the entire feature mapping. To maintain generality, the operation is assumed to have an equal

number of channels for both input and output feature mappings. Consequently, the floating point operations (FLOPs) for PConv are determined only by

$$h \times w \times k^2 \times C_p^2. \quad (1')$$

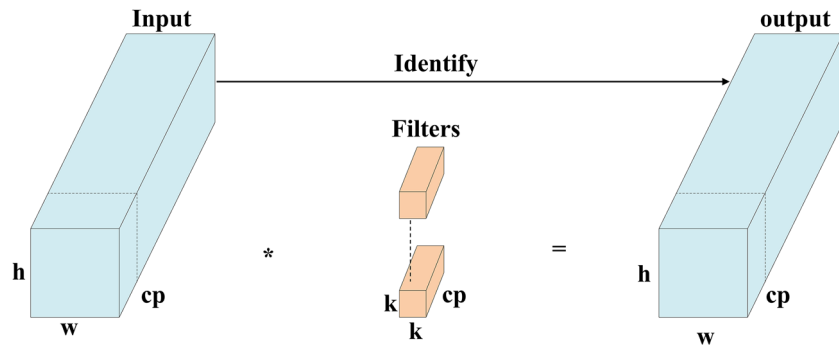


FIGURE 8 | Working principle of Pconv.

At typical bias ratios $r = \frac{Cp}{C} = \frac{1}{4}$, PConv's FLOPs are only $\frac{1}{16}$ of those of a regular Conv. In addition, PConv has smaller memory accesses:

$$h \times w \times 2Cp + k^2 \times Cp^2 \approx h \times w \times 2Cp \quad (2')$$

with $r = \frac{1}{4}$, PConv uses only $\frac{1}{4}$ of the memory compared to a regular convolution, where h is the height of the channel, w is the width of the channel, Cp is the selected continuous network channel, and k is the filter.

2.5.2 | Lw-Detect Module

It is worth noting that throughout the YOLO family, from v3 to v5, the detection head is always coupled. This means that a single convolution layer is used to perform both classification and localization tasks. It was only with the introduction of YOLOX that the YOLO family adopted the decoupled head structure for the first time, aligning it more closely with the principles of advanced detection frameworks (Ge et al. 2021). Following suit, the YOLOv8 model also adopts the decoupled head structure. It extracts category and location features through two parallel branches, and then performs classification and localization tasks with individual 1×1 convolutional layers, as shown in Figure 9a.

Parameter sharing, an important technique in deep learning algorithms, serves to reduce model complexity and improve performance and generalization through parameter sharing. Given the computational constraints of edge devices, we used this approach to construct a lightweight detection head with parameter sharing. The convolutional parameters were made consistent before feature extraction, allowing us to merge the two branches and obtain the initial Lw-Detect module, as shown in Figure 9b. Furthermore, recognizing that the computational overhead of a 1×1 convolution is significantly less than that of a 3×3 convolution, we replaced one of the 3×3 convolutions with a 1×1 convolution, resulting in the improved structure shown in Figure 9c. To minimize the number of network parameters, we went one step further and replaced the standard 3×3 convolution with a 3×3 PConv, as shown in Figure 9d.

Table 1 displays the GFLOPs values for the complete YOLOv8 network with various detection heads. It is evident from the

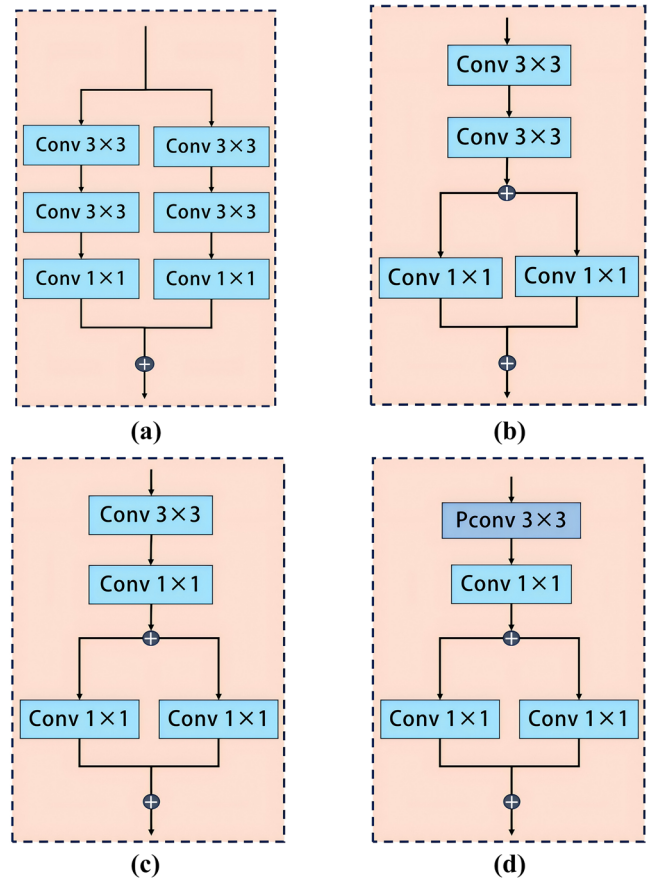


FIGURE 9 | Network structure of the detection head. (a) The original detection head. (b–d) Improved detection heads.

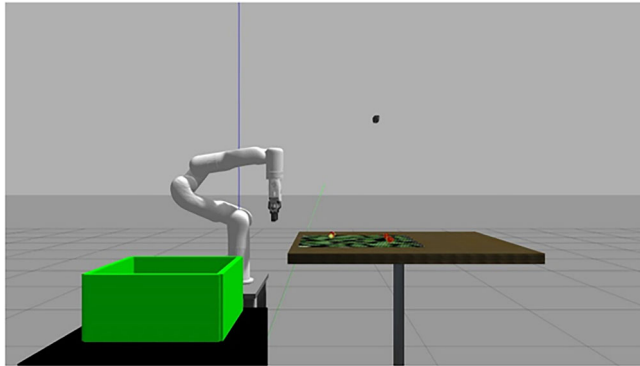
table that the number of parameters in the network model gradually decreases. Specifically, the final detection head D exhibits a reduction of 35.6% in parameters compared to the original model.

2.6 | Robotic Arm Gripping Simulation Experiment

To further validate the practical effectiveness of SP-RTSD, we developed a robotic arm simulation environment within the robot operating system (ROS) framework; the algorithm is quantized by TensorRT. This environment allowed us to

TABLE 1 | FLOPs of four different networks improved based on different Detect modules.

Network	Detect A	Detect B	Detect C	Detect D	FLOPs
A	✓				8.7
B		✓			8.2
C			✓		6.9
D				✓	5.6

**FIGURE 10** | Simulation environment in Gazebo.

perform autonomous strawberry grasping experiments by seamlessly integrating the algorithms with the robotic arm control systems. In our autonomous grasping simulation experiments, we used Ubuntu 18.04 as the operating system, ROS as the robot control platform, Gazebo as the simulation environment, and Moveit software for precise manipulation of the robotic arm. For path planning, we used the RTconnect algorithm (Kuffner and LaValle 2000), which provided efficient trajectory planning. The simulation environment built on the Gazebo platform is shown in Figure 10.

The figure visualizes the six-degrees-of-freedom robotic arm we built for gripping strawberries. Directly to the right of the robotic arm is a green container for the grabbed strawberries, while directly in front of it is a table. Overhead, an RGBD camera is mounted above the table and pointed at a scene of strawberries waiting to be recognized. These strawberries are randomly distributed in the camera's field of view. For the experimental setup, a target strawberry was digitally created using Blender software. The corresponding 3D model and its texture were exported from Blender and then imported into Gazebo to create the simulated strawberry scene. Figure 11 shows the simulation diagram created with Blender.

The whole simulation process can be divided into seven steps: (1) Place the strawberry model randomly in the field of view of the depth camera. (2) Photograph the environment with the depth camera to obtain information about the target object, the strawberry. (3) Obtain the RGB information of the image captured by the depth camera, perform image preprocessing, and use it as the input to the model. (4) Use the TensorRT quantization method to call the trained SP-RTSD model and input the corresponding strawberry position. (5) Obtain the 2D position information (x, y) of the target from the RGB image information, and obtain the depth information from the depth camera, and finally obtain the 3D position

information (x, y, z) of the center of the target. (6) After obtaining the position information of the target, the RTconnect algorithm is utilized to plan the execution path of the robotic arm so that it reaches the corresponding position to execute the grasping operation. (7) If the grasping is successful, the strawberry is transferred to the corresponding position; if the grasping fails, it exits and performs the next grasping.

2.7 | Evaluation Criteria

To assess the enhanced SP-RTSD model, we opted for several common evaluation metrics in target detection. The model's performance is gauged through mean average precision (mAP). mAP signifies the average of the mean accuracy across each class. Before computing mAP, we calculate the average precision (AP) for each class. The principle of AP calculation involves arranging all objects based on their confidence levels, from high to low. Subsequently, accuracy and recall are computed one by one, and the corresponding values are determined. This ratio effectively reflects the overall performance and accuracy of the target detection algorithm. The formula for AP calculation is as follows:

$$AP = \int_0^1 P(R) dR \quad (3')$$

$$mAP = \frac{1}{C} \sum_{i=1}^C AP_i \quad (4')$$

The formula for calculating the average precision (AP) involves the $P-R$ curve, where $P(R)$ represents the change in accuracy P concerning recall R . Here, C denotes the number of labeling classes. The second evaluation metric is FPS, employed to measure the detection speed. The calculation for FPS is as follows:

$$FPS = \frac{1}{\text{inference time per frame}} \quad (5')$$

where the inference time per frame is the time needed to infer each frame.

3 | Experimental Results and Analysis

3.1 | Experimental Platform and Parameter Configuration

Our network training was conducted on Windows 11 using the PyTorch framework. The main configuration of the workstation

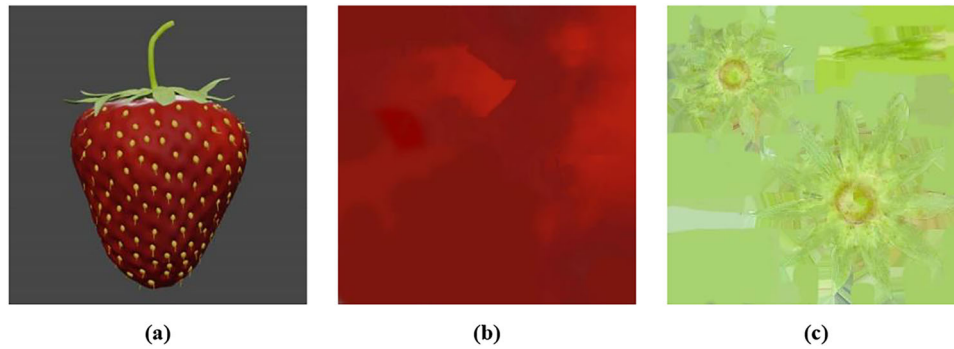


FIGURE 11 | Simulation drawn by blender: (a) strawberry 3D image, (b) strawberry mapping, and (c) root and leaf mapping.

TABLE 2 | PC-based experiment results.

Model	LAWDS	AIFI	mAP0.5	Params	GFLOPS	Model size
YOLOv8n			0.907	3151904	8.7	6.2
A	✓		0.915	2820544	8.6	5.6
B		✓	0.909	3079456	8.6	6.1

TABLE 3 | Real-world detection experiment results in Jetson-Nano.

Model	LAWDS	AIFI	Infer times (ms)	FPS
YOLOv8n			48.4534	20.6381
A	✓		97.0511	10.3039
B		✓	53.5415	18.6771

includes an Intel Xeon E5-2670 CPU and an NVIDIA GeForce RTX 4060 GPU. The compilation language used is Python 3.10, and the deep learning framework is PyTorch 2.0. For this experiment, we utilized the official weights of YOLOv8 for training. The training image size was set to 640 pixels $\times$ 640 pixels, with the Epoch setting at 200. The training batch size was set to 16, while all other parameters were maintained at default values.

For the selection of edge devices, Jetson-Nano was specifically chosen as our real-world inspection test platform. We chose edge devices for a real-world experiment due to cost constraints in the context of smart agricultural machinery. Our preference was for small, low-power, and economically feasible AI devices. The Jetson-Nano was chosen as an optimal solution because it aligns with the specified criteria. It is an inexpensive embedded device that was readily available for approximately 100 US dollars at the time of procurement. Additionally, the Jetson-Nano includes all the essential features required for the development of low-cost AI applications and projects. Its affordability, coupled with a robust set of features, makes it an attractive option for our purposes. The device has a wide range of data transfer interfaces, from GPIO to CSI, allowing for easy connections with various sensors. This versatility enables comprehensive support for a variety of AI applications, which is ideal for our agricultural scenario applications. The Jetson-Nano

is a cost-effective solution that offers a wide range of features to meet our deployment objectives. This makes it a suitable test platform for our improved model in practical agricultural settings.

3.2 | Results and Discussion of Lightweighting Improvement Experiment

To evaluate the concrete effects of the LAWDS and AIFI modules, we performed PC-based experiments and real-world detection experiments based on edge devices. The Jetson-Nano is equipped with TensorRT acceleration, which enhances inference speed and makes it a popular choice for practical applications. We leveraged TensorRT acceleration to optimize the deployment process by using C++ as the chosen method for accelerated deployment during our model deployment onto the Jetson-Nano. The results of the experiments are presented in Tables 2 and 3.

Table 2 analysis shows that incorporating channel attention mechanisms improves accuracy while reducing the number of model parameters and overall size. This highlights the effectiveness of channel attention mechanisms in enabling the network to dynamically select crucial channels for the task. This adaptive selection effectively reduces the computational burden on the model, making it possible to construct lightweight modules. However, the deployment results presented in Table 3 show a less satisfactory outcome. The inference time experiences a considerable increase of 100.29% and 10.50%, respectively. Regarding the comparison of FPS and mAP results between different algorithms on the edge device, we can visualize it similarly as shown in Figure 12. This result is unexpected, especially considering the integration of the TensorRT acceleration principle. Further investigation is warranted to determine the reasons for the increased inference time during the deployment phase.

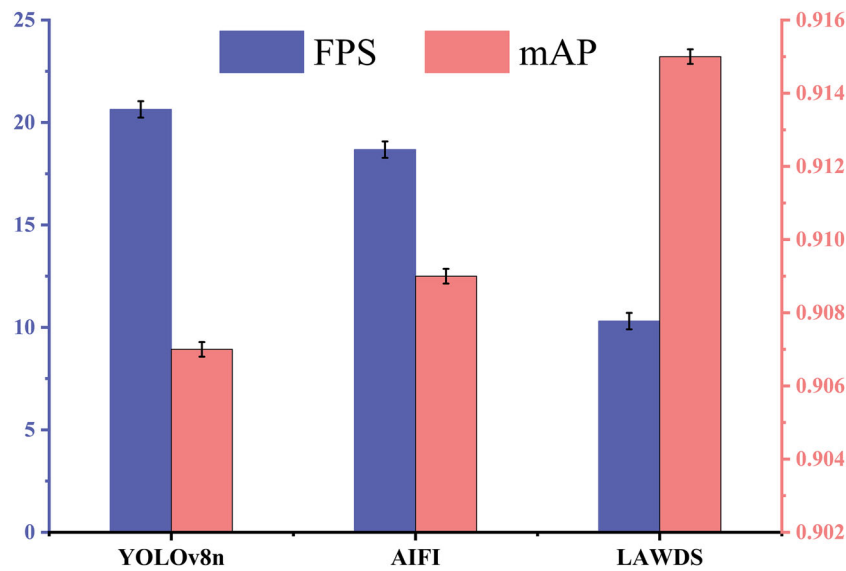


FIGURE 12 | The FPS and mAP results between different improved models.

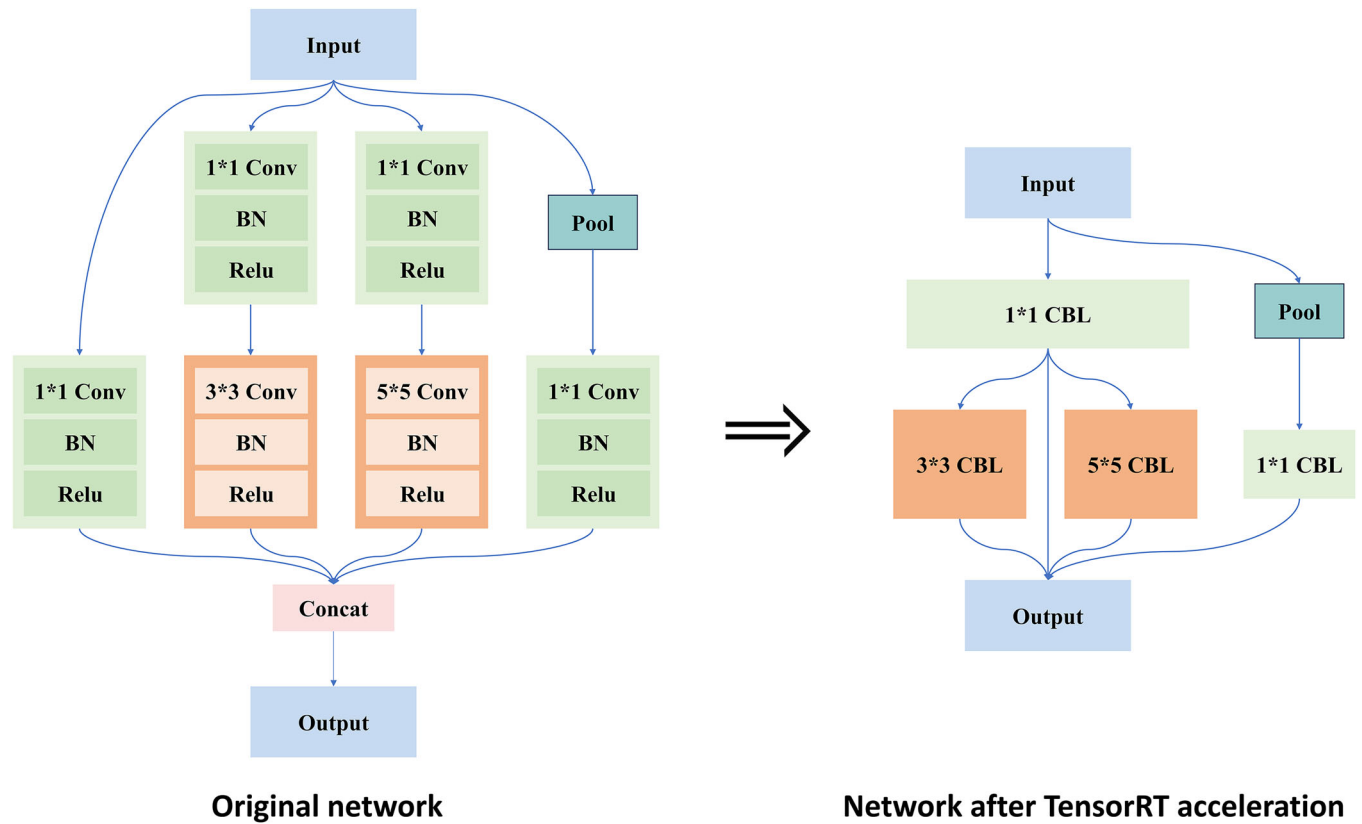


FIGURE 13 | Acceleration principles of TensorRT.

After conducting a thorough investigation, we have determined that the increase in inference time, as shown in Table 3, is due to TensorRT's network restructuring during the acceleration process. This restructuring involves several measures, such as reduction, horizontal integration, and vertical integration. The process of network reorganization is shown in Figure 13. Vertical integration refers to the merging of three key layers in contemporary neural networks, that is, convolutional layer (conv), batch normalization layer (BN), and

rectified linear unit layer (Relu), into a single unified layer called CBL. Horizontal integration combines layers that share the same inputs and perform the same operations. And the removal of the concat layer in the figure is a reduction strategy. Once TensorRT completes the reconfiguration of the network structure, it implements runtime optimizations specifically tailored for GPU hardware. These optimizations improve CUDA cores, refine memory accesses, and implement other relevant measures. This strategic approach ensures

seamless compatibility with NVIDIA GPUs, unlocking the full potential of hardware performance.

However, attention mechanisms introduce additional considerations due to their intricacies. These mechanisms typically assign weighted attention to elements in a sequence, resulting in sequential dependencies. This characteristic poses challenges for certain parallel computation optimizations, as the computation of specific elements may be constrained by dependencies on the results of preceding elements. This understanding clarifies the impact of the channel attention mechanism on Jetson-Nano. The limitations in optimizing parallel computation, combined with the sequential dependencies inherent in attention mechanisms, contribute to the subtle improvements observed in Jetson-Nano. Similarly, the LAWDS module was designed to include convolutional branches in addition to the attention mechanism. However, the inclusion of these convolutional branches conflicts with the horizontal integration in the acceleration strategy. As a result, the LAWDS module has a longer inference time than the AIFI module.

3.3 | Comparison With State-of-The-Art Models

Building upon the insights gleaned from our lightweighting improvement experiment, we devised the SP-RTSD network. To further assess the performance of this enhanced model on the same data set, we performed comparisons with several mainstream detection networks. These include FastestDet, YOLOv5, YOLOv7 and YOLOv8, along with other YOLOv8 enhancement approaches such as RepViT (Wang et al. 2023a) as a replacement for the YOLOv8 backbone network and EfficientViT (Cai et al. 2022) as an alternative YOLOv8 backbone network. The rationale behind integrating two lightweight ViT variants, RepViT and EfficientViT, into the YOLOv8 backbone and replacing them for comparative experiments is as follows: Visual transformers (ViTs) provide an efficient solution for learning visual representations and have demonstrated superior performance compared to CNNs in several computer vision tasks. However, ViT models are often characterized by large size and high latency, making them unsuitable for resource-constrained mobile devices. Consequently, researchers have focused on designing lightweight variants of ViTs. A number of efficient design principles have been proposed, the most notable of which are the RepViT and EfficientViT models. Both of these models significantly enhance the computational efficiency of ViT for edge device deployment. Table 4 shows the detection data for these algorithms.

The table shows that our model outperforms the majority of mainstream models in terms of accuracy. FastestDet stands out with the smallest number of parameters and the most compact model size, due to the single lightweight sensor head and the anchor-free strategy developed by FastestDet. However, in terms of accuracy, FastestDet falls short by only 73.5%, making it less suitable for real-world application requirements. The YOLOv5 network has a similar number of parameters and model size to our model, but shows significantly lower accuracy. Considering all aspects, our model proves to be a superior choice. RepViT shows slightly better accuracy than our model, but its number of parameters and model size are almost four

TABLE 4 | The experimental comparisons of the different detection algorithms.

Model	mAP0.5	Params	Model size
FastestDet	0.735	236944	1.2
YOLOv5	0.902	1763224	3.9
YOLOv7	0.914	6020400	13.2
YOLOv8	0.907	3151904	6.2
YOLOv8+RepViT	0.920	6738041	14.0
YOLOv8+EfficientViT	0.912	4011897	8.7
Our method	0.917	1749712	3.7

times larger than our model, making it unsuitable for edge devices. The remaining models do not match our model in terms of accuracy, number of parameters, and model size.

3.4 | Network Improvement Ablation Experiments

Ablation experiments are commonly used in complex neural networks to explore the impact of specific substructures, training strategies, and parameters on the model. To assess the performance of the C2f-Faster and Lw-Detect modules, we performed ablation experiments and the results are presented in Table 5.

Table 5 shows that our designed modules improve the accuracy of the original model while significantly reducing its size. This improvement makes it easier to deploy the model on edge devices and highlights the efficiency of our proposed modifications. Moving from the A model to Model B, we see a 1.1% increase in accuracy and a 19.54% reduction in the number of model parameters. This is in line with the inherent benefits of Pconv in the C2f-faster module, demonstrating superior feature extraction capabilities and a more streamlined parameter count compared to standard convolution. Continuing this trend from Model B to D, a strategic compromise between accuracy and parameter count is achieved, resulting in a 27.14% reduction in parameters while maintaining acceptable accuracy. This supports the intentional parameter sharing and strategic replacement of 3*3 convolutions with 1*1 convolutions in our detection head design. The enhancements from Model D to E resulted in a 0.5% increase in accuracy and a 27.45% decrease in parameters, demonstrating the positive impact of integrating Pconv into our architecture. The final model's improvements resulted in a remarkable 1% increase in accuracy and an impressive 56.3% reduction in the number of parameters compared to the original model. These results unequivocally validate the efficacy of our designed modules, affirming their proficiency in simultaneously improving accuracy and reducing the model's complexity.

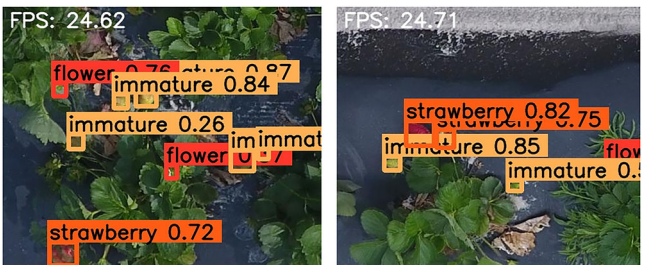
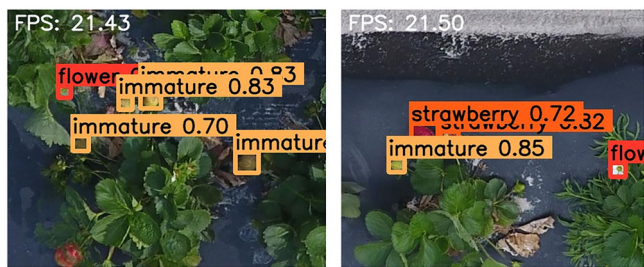
The performance outcomes of the accelerated deployment are presented in Table 6, along with a graphical representation of the deployment results for different models on the Jetson-Nano in Figures 14–18. These figures provide a visual representation

TABLE 5 | Ablation experiment results of SP-RTSD in PC.

Model	C2f-faster	Detect A	Detect B	Detect C	Detect D	mAP0.5	GFLOPs
A (YOLOv8n)		✓				0.907	8.7
B	✓	✓				0.918	7.0
C	✓		✓			0.915	6.4
D	✓			✓		0.912	5.1
E	✓				✓	0.917	3.8

TABLE 6 | Ablation experiment results of YOLOv8 in Jetson-Nano.

Model	C2f-faster	Detect A	Detect B	Detect C	Detect D	Infer time (ms)	FPS
A (YOLOv8n)		✓				48.4534	20.6381
B	✓	✓				46.4593	21.5244
C	✓		✓			45.8066	21.8036
D	✓			✓		42.7775	23.3765
E	✓				✓	39.7066	25.1844

**FIGURE 14** | Model A (YOLOv8n original model).**FIGURE 17** | Model D (C2f-faster as well as Detect C were used instead).**FIGURE 15** | Model B (C2f-Faster used instead).**FIGURE 18** | Model E (C2f-faster and Detect D were used instead).**FIGURE 16** | Model C (substitution of C2f-faster and Detect B).

of the quantitative data presented in Table 6, offering a comprehensive overview of the models' performance in real-world scenarios on the chosen platform.

The table presents the performance improvements achieved by deploying our model on Jetson-Nano and leveraging TensorRT acceleration. The original YOLOv8n model achieves an inference speed of 20.64 FPS. After integrating the C2f-Faster module, this speed increases to 21.52 FPS. Our iterative enhancements resulted in a significant improvement in inference time on the Nano, achieving an impressive 39.71 ms through a systematic replacement of the detection head. Comparing the final improved model with the original, we observed a substantial reduction in inference

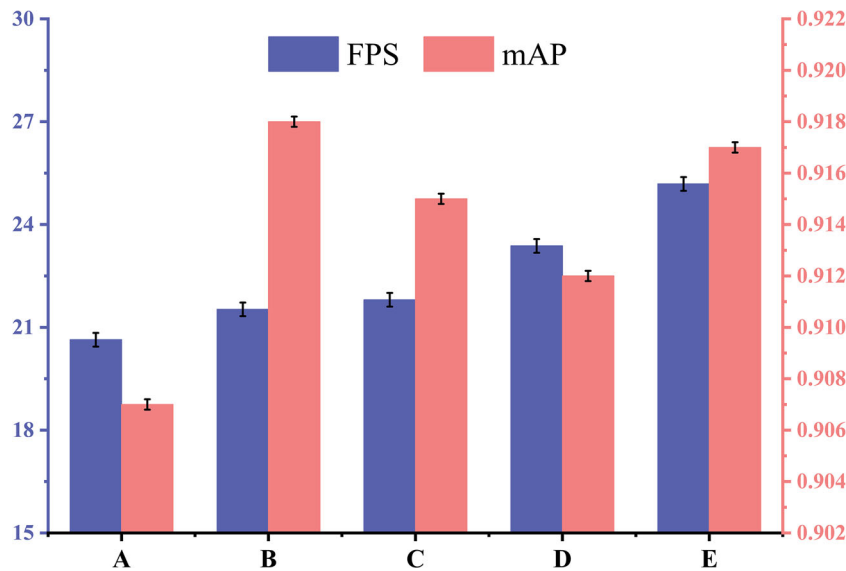


FIGURE 19 | The FPS and mAP results between different improved modules.

time by 9.25 ms, accompanied by an increase in inference speed by 4.55 FPS. The results demonstrate a significant improvement in meeting real-time requirements for strawberry detection. Our model enhancements have optimized performance, proving their capability to meet the stringent demands of real-time applications in the context of strawberry detection. This result is also attributed to the strategic adjustments we made in the design of the C2f-faster and LW-detect modules, where we avoided a multi-branching structure. C2f-faster conforms to the reduction strategy in the TensorRT-accelerated network reorganization process, and LW-detect conforms to the horizontal integration with the vertical integration strategy, which is the main reason why our design module can be successful.

The effectiveness of the lightweight improvements made to our model for edge devices is further substantiated by the graphical representation of our results. The introduction of enhanced modules exhibits a consistent upward trend in inference speed, affirming the success of our optimizations in real-world deployment scenarios. An intriguing observation from the result graphs is the presence of omissions in the outputs of the original model. For example, the left graph of Figure 14 omits strawberries, and the right graph overlooks unripe strawberries. Our final improved result, shown in Figure 18, successfully detects these missed objects. This improvement in detection accuracy highlights the impact of our designed modules on the model's ability to accurately identify and categorize objects. The graphs in the results provide objective evidence of the practical effectiveness of our model improvements. They demonstrate the model's ability to overcome limitations observed during real-world deployment scenarios.

Figure 19 illustrates the FPS and Map results among various modules on the Jetson Nano, thereby demonstrating the efficacy of the designed modules. The final model exhibits a notable enhancement in both FPS and Map compared to the initial model, aligning with the primary design objective of enhancing rate while concurrently improving accuracy.

TABLE 7 | Comparison results of different advanced lightweighting models.

	Model size (MB)	Map0.5	Infer times (ms)
EfficientViT	8.7	0.912	109.008
FasterNet	5.0	0.907	61.7699
Gold-YOLO	12.4	0.917	73.0855
MobileNetV4	11.7	0.896	60.9751
SP-RTSD	3.7	0.917	39.7066

3.5 | Evaluation of Model Performance

3.5.1 | Comparison Experiments With Other Advanced Lightweighting Models

To further demonstrate the superiority of our lightweight models, we conducted a comparative analysis with other state-of-the-art lightweight models. We selected the most representative and high-performing models from recent advancements, including: EfficientViT, designed specifically for mobile devices and embedded systems with low computational complexity and memory requirements; FasterNet, which focuses on neural network architectures with exceptionally high inference speeds, making it ideal for real-time applications; Gold-YOLO (Wang et al. 2023b), the latest enhanced lightweight variant of the YOLOv8 series, renowned for its fast processing speed and real-time detection capabilities; and MobileNetV4 (Qin et al. 2024), a series of lightweight CNN architectures developed by Google, optimized for mobile devices and resource-constrained environments. This selection provides a comprehensive benchmark, allowing us to effectively evaluate the strengths of our models in comparison to established alternatives. Similarly, we accelerated these models with TensorRT's C++ and deployed them to Jetson-Nano. We focus on the comparison of model size, accuracy, and inference time, which is shown in Table 7, and the deployment results are shown in Figure 20.

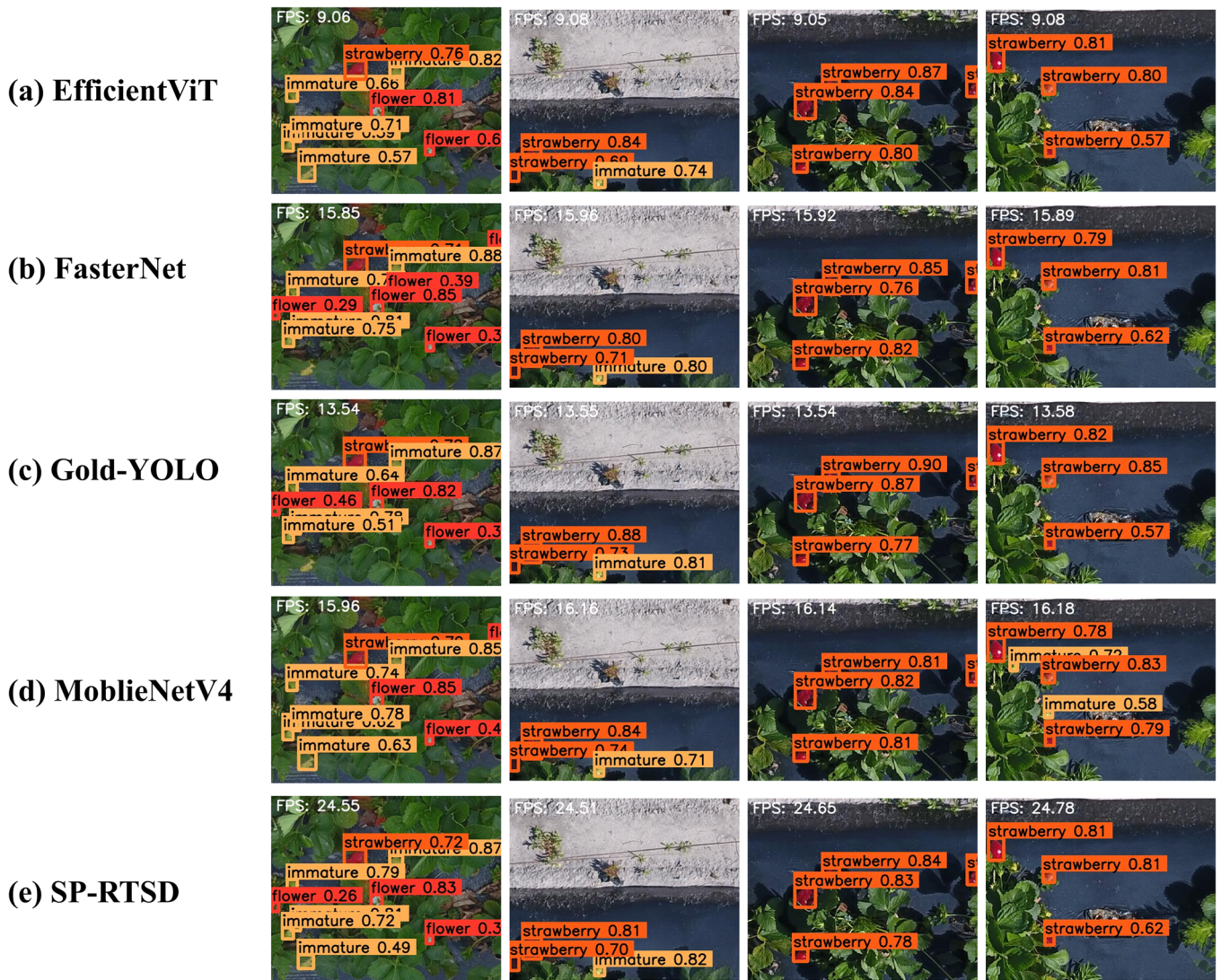


FIGURE 20 | Deployment results of different lightweighting models. (a) EfficientViT, (b) FasterNet, (c) Gold-YOLO, (d) MobileNetV4, and (e) SP-RTSD.

Table 7 shows that our SP-RTSD model exhibits the smallest model size, the highest accuracy, and the shortest inference time compared to other lightweight models. These results indicate that our model is optimally suited for deployment on edge devices. Figure 20 further illustrates the deployment advantages of our lightweight model on edge devices, highlighting its faster inference speed compared to other lightweight models.

3.5.2 | Robustness Test Experiment With Different Strawberry Data

To further verify the robustness and generalization ability of the model, we selected several data sets containing diverse scenes, lighting conditions, and strawberry varieties. More information is available at: <https://github.com/dsgcyj/SP-RTSD.git>. The data sets were primarily sourced from Roboflow (<https://roboflow.com>), an end-to-end computer vision platform offering a large number of open-source data sets. From this, we selected four distinct data sets:

Data set 1: Comprising 1015 images with two labels, “ripe” and “unripe,” used to differentiate between ripe and unripe strawberries.

Data set 2: Contains 522 images with two labels, “morango” and “rotten,” used to distinguish ripe from rotten strawberries.

Data set 3: Consists of 521 images with the same “ripe” and “unripe” labels for distinguishing ripe strawberries from unripe ones.

Data set 4: Contains 7,398 images with three labels, “flower,” “green strawberry,” and “red strawberry,” used to identify strawberry flowers, unripe strawberries, and ripe strawberries, respectively. It is worth noting that almost all detection targets in this data set are occluded.

To highlight the diversity of strawberry morphology, we carefully selected unique samples from the four data sets mentioned above, which posed significant challenges for model identification. As shown in Figure 21, the images in (a)

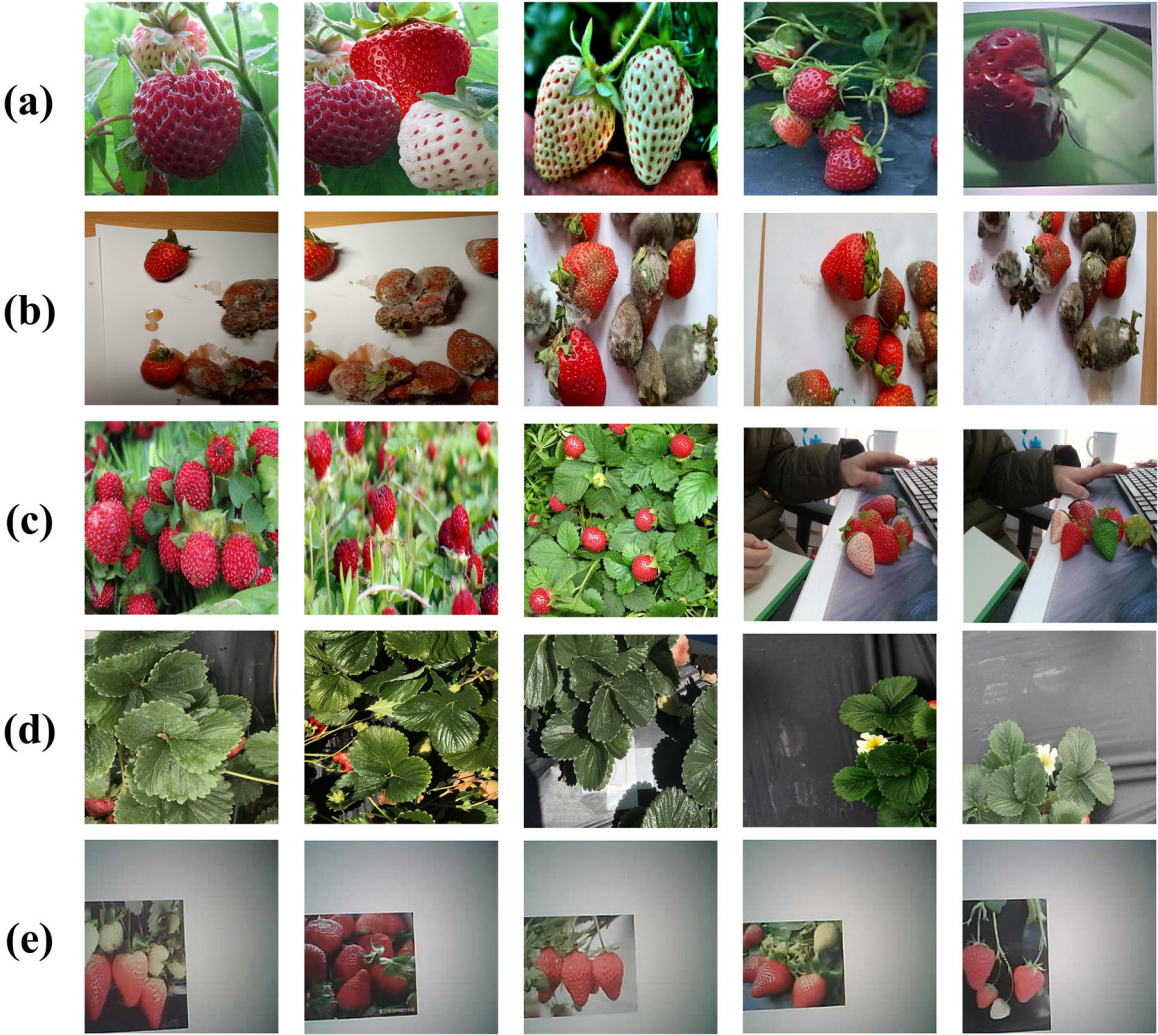


FIGURE 21 | Special strawberry samples in the data set. (a) Other strawberry varieties, (b) Rotting strawberries, (c) Other fruits similar in color and size to strawberries, as well as strawberry models, (d) Strawberries obscured by foliage, (e) Strawberry images.

TABLE 8 | Training results for different data sets.

Data set	Models	Model size (MB)	Map0.5	Infer times (ms)
Data set 1	Baseline	6.3	0.906	47.2945
	SP-RTSD	3.7	0.913	40.2971
Data set 2	Baseline	6.3	0.85	48.0156
	SP-RTSD	3.6	0.883	39.4383
Data set 3	Baseline	6.2	0.828	47.9986
	SP-RTSD	3.6	0.832	40.5089
Data set 4	Baseline	6.3	0.844	47.2766
	SP-RTSD	3.7	0.847	39.1018

feature deep red, round strawberries and white strawberries, which differ from the more common bright red strawberries with pointed bottoms. The images in (b) depict rotting strawberries clustered together in different scenes. (c) contains images of other fruits that are similar in color and size to strawberries, as well as images of non-real strawberry models in different colors. The strawberries in (d) have low visibility,

obscured by various degrees of foliage and shadows. The image in (e) shows a blurred, non-directly captured strawberry. These varied conditions present substantial challenges for current algorithms in accurately detecting ripe strawberries within these data sets. The training of the model on different data sets also uses default parameters, and the training results are shown in Table 8.

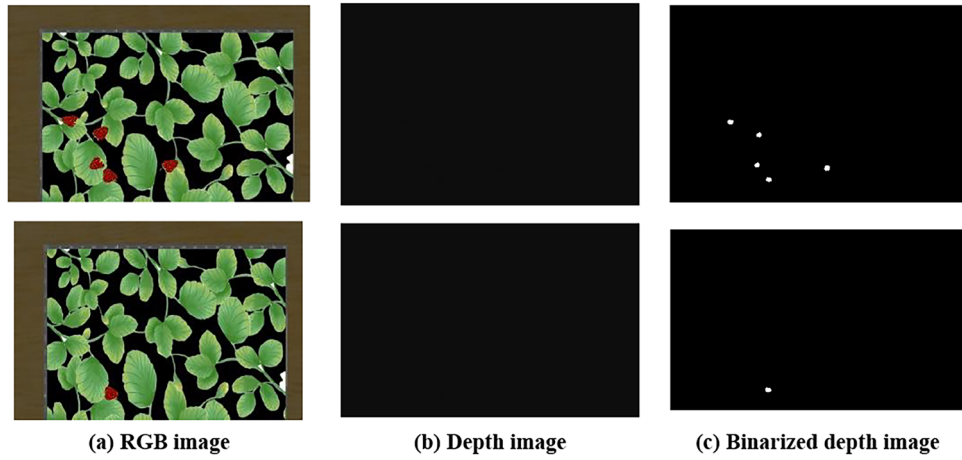


FIGURE 22 | Images acquired by the depth camera. (a) RGB image, (b) depth image, and (c) Binarized depth image.

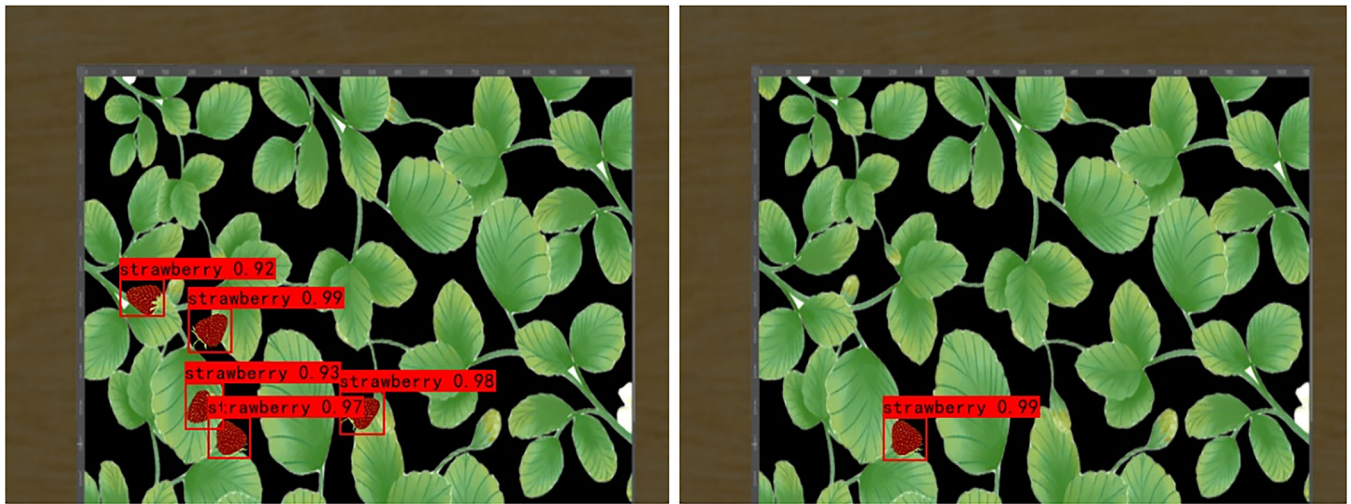


FIGURE 23 | Strawberry detection result.

From Table 8, we observe that across the four different strawberry data sets, the SP-RTSD model we designed demonstrates improvements in both accuracy and inference time. In terms of model size, reductions of 42.16%, 42.86%, 41.94%, and 42.16% were achieved for data sets 1–4, respectively. Regarding Map0.5, improvements of 0.7%, 3.3%, 0.4%, and 0.3% were observed for the same data sets. Additionally, inference times were reduced by 14.80%, 17.86%, 15.62%, and 17.29%, respectively. These results indicate that the enhanced model achieves the expected performance across various strawberry target detection data sets, further demonstrating the robustness and generalization ability of our approach.

3.6 | Experimental Results of Autonomous Gripping by Robotic Arm

In the experiment, the strawberry was randomly positioned within the camera's field of view, and the RGB and depth image data of the target strawberry were captured by the depth camera. The acquired depth image may have lacked sufficient contrast, necessitating the application of a binarization process.

The entire process is depicted in Figure 22, which illustrates the acquired image information.

Subsequently, the acquired RGB image was input into the SP-RTSD model for prediction, resulting in the detection frame of the target. The two-dimensional coordinates of the detection frame's center (x, y) were calculated and illustrated in Figure 23.

Consequently, a combination of the acquired depth information with the coordinate conversion yielded the (x, y, z) coordinate results. The aforementioned results were then input for the subsequent robotic arm grasping step, as depicted in Figure 24. This step involved the robotic arm successfully grasping the strawberry.

The experiments were conducted using both single and multiple strawberries as target scenes, with the strawberries randomly positioned within the camera's field of view. Each experiment was repeated 100 times for each scene to determine the number of successful strawberry detections and grasps. Subsequently, the success rates were calculated. The statistical findings are presented in Table 9.

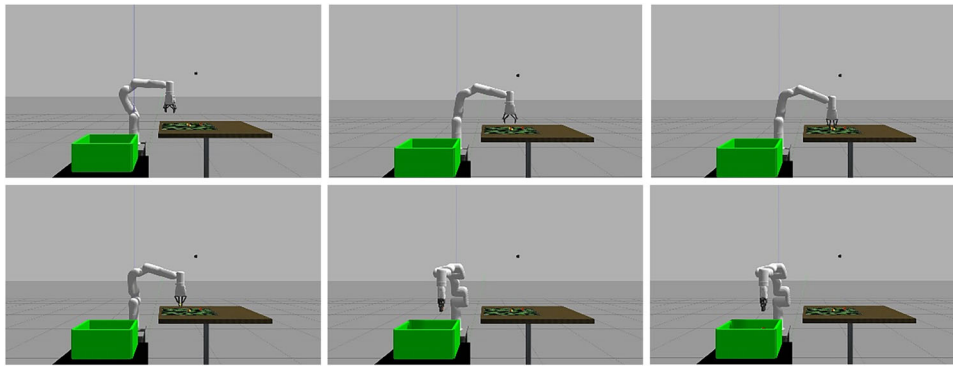


FIGURE 24 | State of the robotic arm during the gripping process.

TABLE 9 | Experimental results of robotic arm simulation gripping.

	Grabs	Successful detections	Successful grabs	Success rate
Single strawberry	100	100	98	98%
Multiple strawberries	100	96	86	86%

A quantitative analysis of the data reveals that the six-degree-of-freedom robotic arm demonstrated superior performance in strawberry grasping, evident in both the single and multiple strawberry scenarios. In particular, in the single strawberry scenario, the robotic arm achieved an impressive success rate of 98%. However, in the context of multiple strawberries, the success rate exhibited a slight decline, reaching 86%. The collective success rate across 100 crawling experiments was 92%, which met the practical application requirements.

4 | Discussion

The integration of a channel attention mechanism and the introduction of a lightweight downsampling structure for lightweight improvement have yielded diverse outcomes when deployed on edge devices. Specifically, the channel attention mechanism, recognized for its weighted attention allocation to sequence elements, introduces sequence dependencies conflicting with parallel computing optimizations in the Jetson-Nano acceleration strategy. This conflict results in an increased inference time. Similarly, the lightweight downsampling structure, featuring convolutional branches, clashes with the flat integration approach in the Jetson-Nano acceleration strategy, leading to a comparable increase in inference time. Conversely, the purposeful design of the C2f-Faster module and the lightweight detection head, integral components in the SP-RTSD architecture, ensures seamless alignment with the Jetson-Nano acceleration strategy. This deliberate alignment is the key reason behind SP-RTSD's achievement of faster and more accurate detection. These findings underscore the critical importance of tailoring deployment strategies to the anticipated benefits of edge device acceleration, offering valuable insights for future research endeavors focused on refining lightweight improvements for edge devices.

SP-RTSD is a specialized algorithm designed for proficient strawberry recognition on edge devices facing resource constraints. The main objective of the research is to achieve

accurate and fast strawberry recognition, and we have successfully addressed the original research question. The results of the study show significant improvements in speed and accuracy, effectively mitigating issues such as leakage and false detections—key factors in the success of robotic strawberry harvesting. This highlights the potential of the SP-RTSD as a promising solution to mitigate the labor-intensive and costly problems associated with modern strawberry harvesting. Furthermore, SP-RTSD demonstrated an impressive combined success rate of 92% in strawberry grasping simulation experiments with a robotic arm, thereby confirming its suitability for integration into practical picking machines. In comparative experiments with other state-of-the-art lightweight models, SP-RTSD outperformed its counterparts in both inference accuracy and speed, significantly surpassing other lightweight algorithmic models. This highlights its sophistication and advanced design. Furthermore, SP-RTSD delivered the expected performance across a variety of strawberry target detection data sets, further demonstrating its robustness and adaptability.

Due to cost and product size constraints, intelligent agricultural machines prioritize small, low-power, and low-cost AI edge devices, often using hardware acceleration in practical applications. Existing edge device lightweighting approaches often focus solely on reducing network complexity or building structures with fewer parameters, overlooking the edge device hardware acceleration strategy. This oversight often results in compromised accuracy during deployment. SP-RTSD differentiates itself by taking edge device hardware acceleration into account in its network design, solving the problem of accuracy degradation.

5 | Conclusion

In this study, we propose a lightweight, real-time strawberry detection model called SP-RTSD, specifically designed for edge devices. Several key conclusions are summarized as follows:

(1) Performance of lightweight methods on edge devices: We explored the effectiveness of common lightweight techniques on edge devices and discovered that when these methods conflict with the deployment strategies of edge devices, they may fail. This explains why certain lightweight models perform well on PCs but show counterproductive results when implemented on edge devices.

(2) C2f-Faster module's spatial feature screening: The C2f-Faster module incorporates a spatial feature screening process, effectively focusing on critical regions of interest. This approach addresses challenges like occlusion and shadowing commonly encountered in strawberry harvesting scenes.

(3) Lw-detect detection head: The Lw-Detect detection head, with its parameter-sharing mechanism, reduces the number of parameters without compromising classification or localization performance, making it highly efficient for real-time applications.

(4) Improvement in Accuracy and Inference Speed: Our model achieves simultaneous improvements in both accuracy and inference speed, overcoming the common trade-off between accuracy and speed in edge device deployments.

(5) Superior Performance Compared to Other Lightweight Models: The proposed SP-RTSD model outperforms other lightweight models in terms of accuracy and inference speed on edge devices and demonstrates outstanding performance across various strawberry detection tasks.

However, our experiments have some limitations. While fruit detection is crucial to the picking process, key point detection is also critical for the successful operation of a picking robot. In future research, we plan to incorporate strawberry key point detection techniques, particularly tailored for edge devices, to enhance the efficiency and accuracy of strawberry picking robots.

Acknowledgments

This study was funded by the National Natural Science Foundation of China (NSFC), grant number: 61905219, the Zhejiang Science and Technology Cooperation Plan, grant number: 2024SNJF071, China Agriculture Research System (CARS-01), and National Key Research and Development Program of China (2023YFD1401100).

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

Anraeni, S., D. Indra, D. Adirahmadi, and S. Pomalingo. 2021. "Strawberry Ripeness Identification Using Feature Extraction of RGB and K-Nearest Neighbor." Paper Presented at the 2021 3rd East Indonesia Conference on Computer and Information Technology (EIConCIT).

Bai, Y., J. Yu, S. Yang, and J. Ning. 2024. "An Improved Yolo Algorithm for Detecting Flowers and Fruits on Strawberry Seedlings." *Biosystems Engineering* 237: 1–12.

Cai, H., C. Gan, and S. Han. 2022. "Efficientvit: Enhanced Linear Attention for High-Resolution Low-Computation Visual Recognition." arXiv preprint arXiv:220514756.

Chen, J., S.-h. Kao, H. He, et al. 2023. "Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks." Paper Presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.

Ge, Z., S. Liu, F. Wang, Z. Li, and J. Sun. 2021. "Yolox: Exceeding Yolo Series in 2021." arXiv preprint arXiv:210708430.

He, L., W. Fang, G. Zhao, et al. 2022. "Fruit Yield Prediction and Estimation in Orchards: A State-Of-The-Art Comprehensive Review for Both Direct and Indirect Methods." *Computers and Electronics in Agriculture* 195: 106812.

Kuffner, J. J., and S. M. LaValle. 2000. "RRT-Connect: An Efficient Approach to Single-Query Path Planning." Paper presented at the Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065).

Li, C., J. Lin, Z. Li, C. Mai, R. Jiang, and J. Li. 2024. "An Efficient Detection Method for Litchi Fruits in a Natural Environment Based on Improved YOLOv7-Litchi." *Computers and Electronics in Agriculture* 217: 108605.

Lin, G., Y. Tang, X. Zou, J. Cheng, and J. Xiong. 2020. "Fruit Detection in Natural Environment Using Partial Shape Matching and Probabilistic Hough Transform." *Precision Agriculture* 21: 160–177.

Lv, W., S. Xu, Y. Zhao, et al. 2023. "Detrs Beat Yolox on Real-Time Object Detection." arXiv preprint arXiv:230408069.

Pang, F., and X. Chen. 2023. "MS-YOLOv5: A Lightweight Algorithm for Strawberry Ripeness Detection Based on Deep Learning." *Systems Science & Control Engineering* 11, no. 1: 2285292.

Qin, D., C. Lechner, M. Delakis, et al. 2024. "MobileNetV4-Universal Models for the Mobile Ecosystem." arXiv preprint arXiv:240410518.

Redmon, J., S. Divvala, R. Girshick, and A. Farhadi. 2016. "You Only Look Once: Unified, Real-time Object Detection." Paper Presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.

Rizon, M., N. A. N. Yusri, M. F. A. Kadir, A. R. bin Mamat, A. Z. Abd Aziz, and K. Nanaa. 2015. Determination of Mango Fruit From Binary Image Using Randomized Hough Transform. Paper presented at the Eighth International Conference on Machine Vision (ICMV 2015).

Wang, A., H. Chen, Z. Lin, and P. H. Ding G. 2023a. Repvit: Revisiting Mobile cnn From vit Perspective. arXiv Preprint arXiv:230709283.

Wang, C., W. He, Y. Nie, et al. 2023b. "Gold-YOLO: Efficient Object Detector via Gather-And-Distribute Mechanism." *Advances in Neural Information Processing Systems* 36: 51094–51112.

Wang, D., and D. He. 2021. "Channel Pruned Yolo V5s-Based Deep Learning Approach for Rapid and Accurate Apple Fruitlet Detection Before Fruit Thinning." *Biosystems Engineering* 210: 271–281.

Zhang, Y., J. Yu, Y. Chen, W. Yang, W. Zhang, and Y. He. 2022. "Real-Time Strawberry Detection Using Deep Neural Networks on Embedded System (Rtsd-Net): An Edge Ai Application." *Computers and Electronics in Agriculture* 192: 106586.

Zhou, B., X. Lin, J. Zhou, Y. Wang, and F. Hu. 2023. "An Improved Lightweight Parameters Network for Strawberry Flowers Detection." *IEEE Access* 11: 63761–63772.